Getting past the login form

I first tried a couple bruteforces using the most common usernames and passwords from SecLists, however when none of these worked, I had to try some other way of finding the correct credentials.

I googled “dvwa default credentials” and quickly found that they are “admin” and “password”. Trying this logged me in as the admin user on the website.

Finding the flag on the website

SQL injection

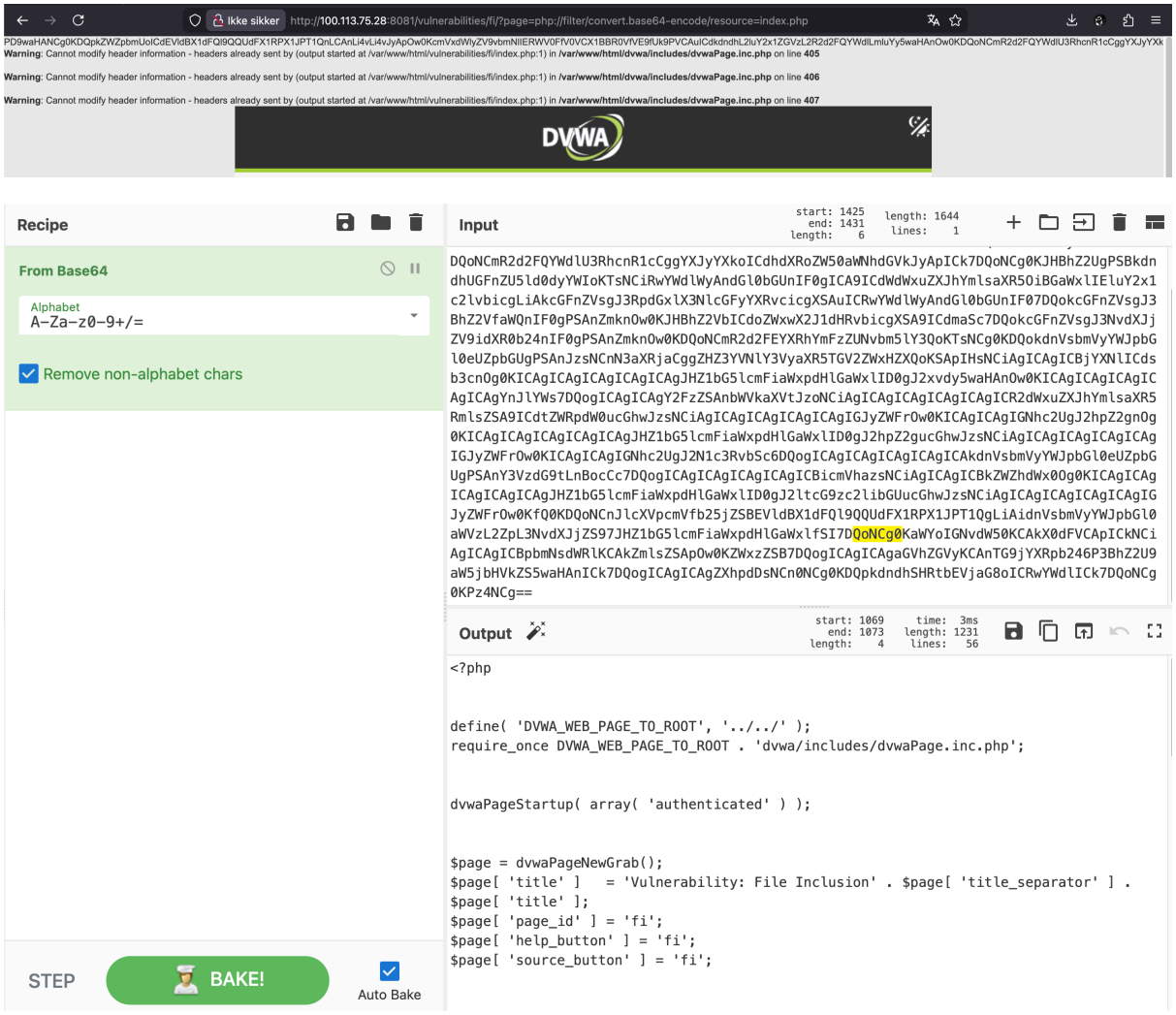
I first tried to see if I could find any SQL vulnerabilities on the SQL injection path. However, I was not able to find any vulnerable parameters, so I decided to move on.

HTTP Open Redirect

On HTTP Open Redirect, I found that each info page had it’s own id, so I tried to fuzz a lot of ids to check if any of them had any hidden information that I could further use. However, with no hit on ids 1-10000 (other than the two / three already available) I decided to move on from this as well, as I was not able to find any reasonable way to exploit this.

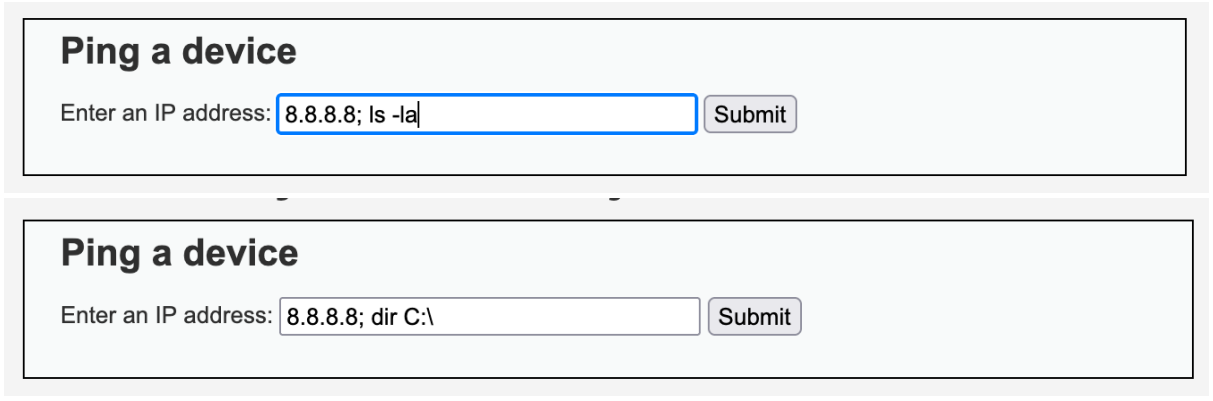
File Inclusion

File inclusion is probably the place that gave me the most hope without actually having any success. By accessing a certain url (seen in the picture below), I was able to find the index.php source code in base64 format. By decoding it in cyberchef, I was hoping to find some interesting hidden paths or information I could use for further exploiting, however this was not the case. Hence, I decided to move on from this as well.



Command Injection

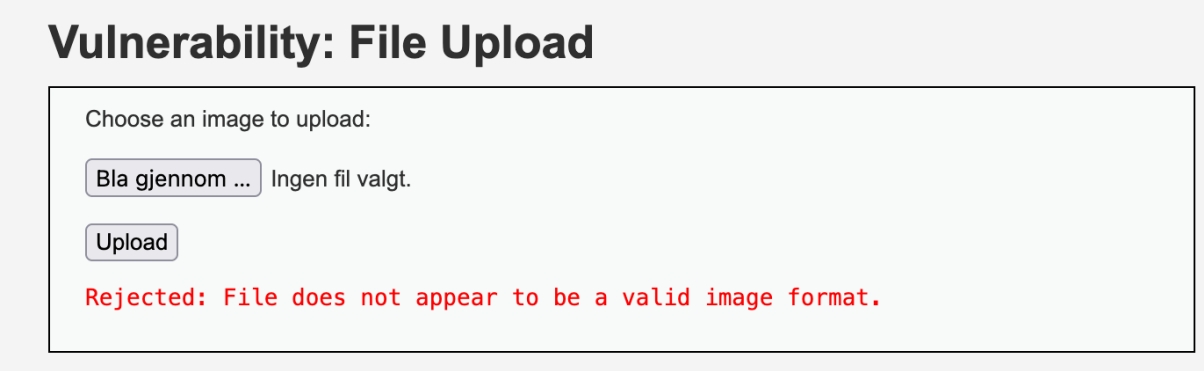
I tried different types of commands, and separators for the commands however it gave no results. I also tried commands for different OS'es, but with no result at all from any form of vulnerability check, I decided to move on.



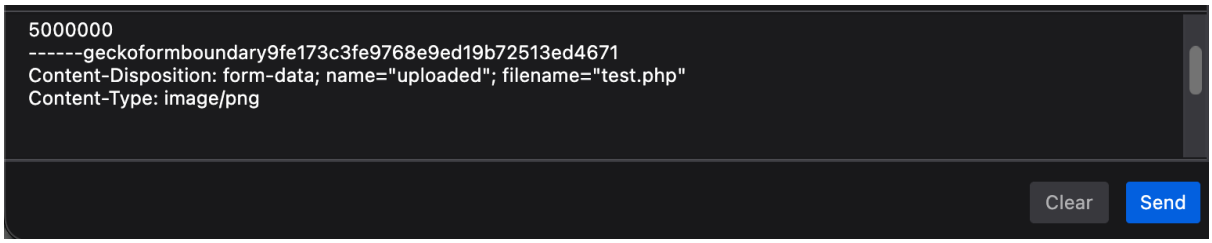
File Upload

File upload is where I had the most success, and was able to test different kinds of exploits with different results each time, giving me a lot of hope that this could be a vulnerable place to test.

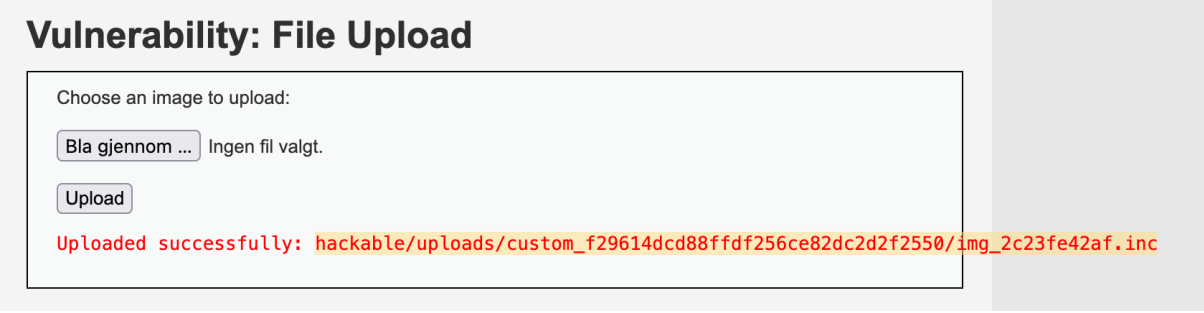
The first I tried was to upload a simple php file, to test if I could just inject a shellcode I could access on the page and then use to execute codes through the URL. However, as seen from the image below, the website did not accept files that are not valid image formats.



Because of this, I decided to try to change the MIME of the file to one that corresponds to an actual MIME of an image. By doing this I got a different error message saying that the file ending was not allowed to upload on the website, making me even more hopeful that this could end up working if I could the correct exploit working.

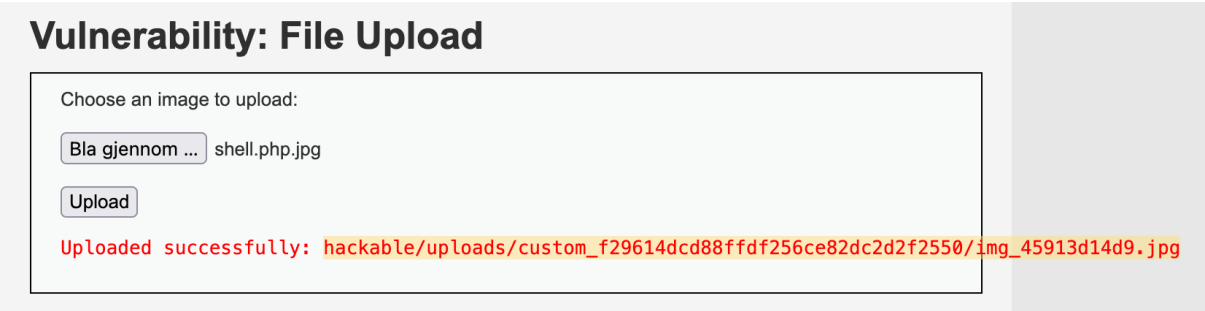


The next thing I decided to try, was to change the .php file ending to a .inc file ending, since I from research found that this file ending could sometimes be used to execute php code. However, when I uploaded the file, and tried accessing the website, it just downloaded the file instead of letting me use the URL to execute the code, indicating that this bypass did not work.

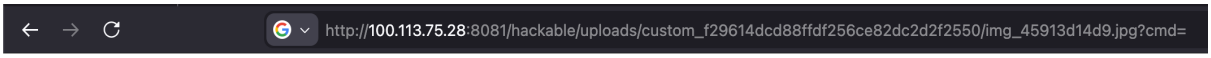


I then decided to try a different file ending bypass, where you use two file extensions for one file, since most file extensions checks only checks the trailing file extension. By

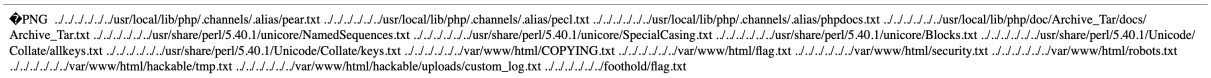
uploading the code with a .php.jpg file extension, I was able to access the site and change the URL to include any command I wanted.



By accessing the site and giving the below command I was able to find all files within the web system that was a .txt file, considering I was looking for a file containing the flag.

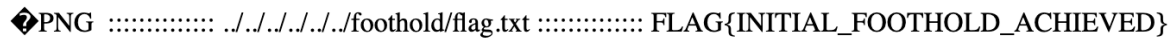


find ../../../../../.. -name "*.txt" -type f



Here we see two potential candidates to hold the flag, and by using the “more” command, which is another way of reading content of a file other than “cat” which did not work on this system, I was able to check the content of both the files. The first file within www/html/ was empty, however when checking the one within /foothold/, I was able to retrieve the flag:

more ../../../../../../foothold/flag.txt



FLAG: FLAG{INITIAL_FOOTHOLD_ACHIEVED}